

1 Introduction

1.1 Project Title

Nogor Shomadhan — *A Mobile Platform for Community Maintenance Issue Reporting, AI-Assisted Triage and Resolution Tracking.*

The name *Nogor Shomadhan* translates from Bengali as “City Solution”, reflecting the aim of the project: turning scattered, informal complaints about a neighbourhood into an organised, traceable and measurable resolution process.

1.2 Project Overview

Residents of a residential community routinely face local infrastructure and maintenance problems — damaged roads, faulty streetlights, water leakage, overflowing garbage bins, blocked drainage, broken park facilities and security-related concerns. In most communities these problems are still reported through phone calls, social media groups or informal messaging, so complaints arrive through disconnected channels, duplicate reports pile up, evidence is missing, and residents have no way to tell whether anything is being done.

Nogor Shomadhan replaces that informal process with a single mobile application that carries a complaint through its entire life cycle. A resident submits a report containing a title, a written description, a structured location (house, road, avenue and nearby landmark) and photographic evidence. A generative-AI model then reads the submitted text together with the attached photographs and automatically assigns the complaint to one of twelve civic categories, so the resident is never asked to classify the problem manually. The same AI layer immediately compares the new report against recent complaints in the same category and, when it recognises the same real-world problem, raises a duplicate flag together with a confidence score and a written justification for a human reviewer, instead of silently discarding the report.

An administrator verifies the complaint, resolves any duplicate flag, and forwards genuine complaints to the community authority. The authority manages the complaint from a dedicated dashboard: it accepts the work, assigns a contractor, sets and revises deadlines, posts progress updates with photographic evidence, and finally marks the complaint resolved. Every status transition is recorded in an immutable history, so the resident sees a complete, timestamped timeline of what happened rather than a single opaque status label. Once the issue is closed the resident rates the quality of the resolution and can leave written feedback, which the authority answers from a feedback centre. Any resident may also raise the urgency counter on an existing complaint, letting the community collectively signal which problems need attention first.

Around this core workflow the platform provides a role-aware notification system that is generated inside the database itself, so residents, authority staff and administrators are informed of every relevant event; a community forum where authorities publish announcements, updates and alerts and residents hold moderated discussions; analytics dashboards that visualise complaint distribution by category and by area, monthly trends, resolved-versus-pending ratios and resolution-time statistics; exportable PDF reports generated on the device; and an administrative activity log and system-settings panel that make platform-level actions auditable and configurable.

1.3 Target Users

The platform is built around three primary, clearly separated user roles. Each role is a distinct route group in the application with its own navigation, permissions and dashboard, so a user only ever sees the screens and data relevant to their responsibility.

Table 1: Target users of the Nogor Shomadhan platform

User Role	Who They Are	What They Do on the Platform
Resident	Verified residents of the community (flat owners and tenants)	Register an account for approval; submit complaints with photographs, a structured location and a description; track the status timeline of their own reports; browse and comment on the community complaint feed; raise the urgency counter on existing complaints; rate and give feedback on resolved issues; participate in the community forum; view personal analytics and export PDF reports.
Community Authority	Staff of the community management or maintenance office	Review complaints forwarded by the administrator; accept or reject work; assign contractors; set and revise deadlines; post progress updates with evidence photographs; move complaints through the <i>Pending</i> → <i>In Progress</i> → <i>Resolved</i> life cycle; respond to resident feedback from the feedback centre; publish official announcements, updates and alerts to the forum; monitor operational analytics and resolution performance.
Administrator	Platform owner or community management committee representative	Approve or reject resident and authority account registrations; verify submitted complaints before they reach the authority; adjudicate AI-flagged duplicate reports; moderate forum content; configure system settings such as AI auto-categorisation, duplicate-detection sensitivity and maintenance mode; review the platform-wide activity log and system analytics.

Beyond these three operational roles, the platform also serves the community management committee as a secondary, indirect audience: the analytics dashboards and exportable PDF reports turn the accumulated complaint history into evidence for budgeting, contractor evaluation and long-term maintenance planning. Finally, a set of public, unauthenticated pages (Home, About, Impact and Contact) addresses prospective residents and visitors, communicating what the platform does before they register.

5 Technologies and Integrations

This section documents the frameworks, external services and AI models used to build *Nogor Shomadhan*, and explains why each was selected over the alternatives that were considered. The guiding constraints were that the system had to be delivered by a small student team within a single semester, run on real Android devices as well as in a browser for demonstration, and

support a relational, audit-heavy data model without the team having to build, deploy and maintain a separate backend server.

5.1 Technology Stack Overview

Table 2 lists the technologies that make up the platform, grouped by the layer of the system they serve. Each of them is described, and its selection justified, in the subsections that follow.

5.2 Frontend Framework: Expo and React Native

The application is built with **Expo SDK 57** on top of **React Native 0.86** and **React 19**, written entirely in **TypeScript** with `strict` type checking enabled. Routing is handled by **Expo Router 57**, which maps the directory structure under `src/app/` directly onto application routes.

Expo Router’s route groups are used to express the role architecture of the system: `(public)`, `(resident)`, `(authority)` and `(admin)` each form an isolated navigation tree with its own `_layout.tsx` file that acts as an access guard for that role. A resident can therefore never reach an administrator screen through navigation, because the guard lives at the layout level rather than being repeated inside every screen. Typed routes were enabled so that an incorrect navigation target becomes a compile-time error instead of a runtime dead end.

Justification. Three alternatives were considered for the client application:

- **Native Android (Kotlin) and iOS (Swift) development** would have required two separate codebases, two toolchains and two sets of UI work for the same feature set — not feasible for a four-member team within one semester.
- **Flutter** offers comparable cross-platform reach, but would have required the team to adopt Dart and would have cut the project off from the JavaScript ecosystem that the Supabase and Google Generative AI client libraries are written for. The team’s existing competence in JavaScript and TypeScript made React Native the lower-risk choice.
- **Bare React Native** (React Native CLI without Expo) would have given the same rendering model, but the project depends on more than twenty native modules — camera and gallery access, secure storage, file system, printing, sharing, gesture handling, animation worklets. Under bare React Native each of these requires manual native linking and per-platform build configuration. Expo’s config plugins and prebuild pipeline handle that automatically, which removed a large and unforgiving class of build problems from the project.

Expo was additionally preferred because **React Native Web** is part of the same toolchain: the identical codebase produces a browser build, which made in-class demonstration and day-to-day development markedly faster than launching an emulator for every change. Expo Router was chosen over configuring **React Navigation** manually because file-based routing keeps the navigation structure visible in the file tree and makes role-based guarding a structural property of the project rather than a convention the team has to remember to apply.

Table 2: Summary of the technology stack

Layer	Technology	Role in the Project
Application framework	Expo SDK 57 with React Native 0.86 and React 19	Cross-platform mobile application for Android and iOS, plus a web build for demonstration
Language	TypeScript 6 (strict mode)	Statically typed application code, with shared domain types derived from the ER diagram
Navigation	Expo Router 57 (file-based routing, typed routes)	Route groups per user role with per-group layout guards
Backend platform	Supabase (PostgreSQL, Storage, PL/pgSQL, pg_cron)	Database, file hosting, business-rule triggers and scheduled jobs
Data access	@supabase/supabase-js v2	Typed client-side queries, filtering and mutations against PostgreSQL
Artificial intelligence	Google Gemini Flash via @google/generative-ai	Multimodal complaint categorisation and duplicate-report detection
Messaging integration	BulkSMSBD HTTP SMS API	Out-of-band SMS notices for account approval and rejection
Device capabilities	expo-image-picker, expo-image, expo-file-system, expo-print, expo-sharing, expo-secure-store	Evidence capture, image rendering, on-device PDF report generation and sharing
UI and interaction	React Native Reanimated 4, Gesture Handler, Safe Area Context, Keyboard Controller, @expo/vector-icons	Animated, gesture-driven and accessible interface components
Local persistence	@react-native-async-storage/async-storage	Session persistence, cached notification state and offline-tolerant preferences
Tooling	Expo CLI, Metro bundler, ESLint (eslint-config-expo), Git	Development server, bundling, static analysis and version control

5.3 Backend and Database Platform: Supabase

Supabase provides the entire server side of the platform. Four of its capabilities are used.

PostgreSQL database.

The relational schema implements the ER diagram directly: `account`, `complaints`, `evidence`, `duplicate`, `complaint_status_history`, `complaint_work_updates`, `contractor_history`, `feedback`, `forum posts and comments`, `notifications` and `app_settings`. Domain vocabularies such as account status, account role, complaint status, complaint category and notification type are declared as PostgreSQL `ENUM` types, so invalid values are rejected by the database rather than merely discouraged by the client. Row Level Security is enabled on every table with explicit access policies.

Storage.

Evidence photographs submitted by residents and progress photographs uploaded by the authority are stored in Supabase Storage buckets. Images captured through the picker are decoded from base64 into a binary buffer before upload, and the resulting public URL is written into the `evidence` table alongside the complaint.

PL/pgSQL functions and triggers.

The notification system is implemented inside the database. Triggers on account insertion, complaint status changes, duplicate insertion, duplicate adjudication, work updates, feedback, feedback replies, forum comments and official announcements call a shared `enqueue_notification` routine that writes role-addressed notification rows carrying a deduplicating event key, a priority level and a deep-link action path.

Scheduled jobs.

The `pg_cron` extension runs time-based notification generators inside the database — for example, flagging a complaint that has remained pending for fourteen days without work being started, and raising deadline-milestone and overdue notices.

Justification. Supabase was chosen over the two obvious alternatives:

- **A custom backend (Node.js/Express with PostgreSQL)** would have given complete control, but the team would then have had to design, implement, document, deploy and keep a REST API online in addition to building the mobile client. Supabase exposes PostgreSQL through a generated, policy-enforced client SDK, so the endpoints planned in the API section of this report are satisfied by typed queries from the application's service layer, with no server code to host. This removed an entire deployment surface from a semester-length project.
- **Firebase (Firestore)** was rejected because the data model is strongly relational. The platform constantly joins complaints to accounts, evidence, status history, contractor history, feedback and duplicate records, and the analytics screens aggregate across those joins by category, area, month and resolution time. A document store would have forced denormalisation and client-side aggregation, and it offers no equivalent of `ENUM` domains, foreign keys or transactional triggers. PostgreSQL expresses the ER diagram as designed.

Implementing notifications as database triggers rather than as client-side code or separate cloud functions was a deliberate decision: a notification row is created in the same transaction as the state change that caused it, so notifications cannot drift out of step with the data, and they are produced correctly no matter which role or which screen triggered the change. For the same reason `pg_cron` was preferred over an external scheduler or worker process — the schedule lives next to the data it inspects, with no additional service to keep running.

5.4 AI Models and APIs

Artificial intelligence is used for the two smart features of the platform, both served by **Google Gemini Flash** through the official `@google/generative-ai` JavaScript SDK.

5.4.1 Automatic Complaint Categorisation

When a resident submits a complaint, the title, the description and the attached evidence photographs are sent to Gemini in a single multimodal request. The model is constrained to reply with exactly one label from a fixed list of twelve categories — Road Damage; Garbage & Waste; Drainage & Waterlogging; Streetlight & Electrical; Water Supply; Sanitation & Public Toilets; Traffic & Illegal Parking; Public Safety & Encroachment; Noise & Environmental Pollution; Parks & Public Spaces; Animal-Related Issues; and Other. The returned label is validated against that list before it is accepted, and the system falls back to *Other* if the response is unrecognised or the service is unavailable, so complaint submission never fails because of the AI layer.

5.4.2 Duplicate Complaint Detection

Duplicate detection runs as a two-stage pipeline:

1. **Database prefilter.** A PostgreSQL query retrieves a small candidate set — the most recent complaints in the same category, excluding the new complaint itself. This keeps the AI request small and cheap regardless of how large the complaint history grows.
2. **AI adjudication.** The new complaint and the candidate set are passed to Gemini, which is instructed to reason about physical location (house, road, avenue, landmark), category and timing rather than mere word overlap, and to return a strict JSON verdict containing a boolean duplicate decision, a confidence score, the matched complaint identifier and a written reason. When the score crosses the configured threshold, a row is inserted into the `duplicate` table carrying the score and the model’s reasoning, which in turn raises a duplicate-review notification for the administrator.

Crucially, the AI never deletes a complaint. It produces evidence; an administrator makes the decision on a dedicated review screen. The sensitivity of the check is configurable by the administrator from the system-settings page, and both AI features can be switched off entirely through the `app_settings` table.

Justification. Several approaches were evaluated:

- **Training a custom classifier** (for example a fine-tuned BERT model or a classical TF-IDF classifier) would have required a labelled corpus of Bangladeshi community complaints. No such dataset exists, and collecting and annotating one was outside the scope of the project. A general-purpose instruction-following model achieves usable accuracy with no training data at all.
- **Keyword matching or TF-IDF cosine similarity** for duplicate detection was rejected because the reports it must compare are written by different residents in free text, frequently mixing Bengali and English and describing the same pothole in entirely different words. A lexical measure misses paraphrase, and it cannot weigh location and recency the way the task requires.
- **Vector embeddings with pgvector** were a serious candidate and would scale better on a very large corpus. They were not chosen because a similarity score alone cannot explain itself: the administrator review screen needs a human-readable justification for why two complaints were judged to be the same problem. A single generative call returns the score *and* the reasoning, which made the review workflow possible without building a second explanation mechanism.
- **Other commercial models** (such as OpenAI GPT-4 class or Anthropic Claude models) are comparable in capability. Gemini Flash was selected because its free tier is sufficient for a student project, it accepts text and images in the same request — which matters because the photograph often identifies the category better than the resident’s wording — and its latency is low enough to run inside the complaint submission flow without making the user wait noticeably.

5.5 External APIs and Third-Party Services

BulkSMSBD SMS gateway. Account approval and rejection decisions are delivered to the applicant by SMS through the BulkSMSBD HTTP API. This closes a genuine gap in the workflow: a resident whose registration is still awaiting approval cannot sign in, and therefore cannot receive an in-app notification. A channel outside the application is the only way to reach them. BulkSMSBD was chosen over international providers such as Twilio because it supports Bangladeshi eleven-digit mobile numbers natively with local operator routing, its pricing suits a local deployment, and it requires no international sender registration. Email was considered as a cheaper alternative but rejected because SMS reach and read rates are substantially higher among the intended user base.

On-device PDF report generation. Analytics reports and complaint reports are produced with `expo-print`, which renders an HTML document generated by the application into a PDF, and `expo-sharing`, which hands the resulting file to the operating system share sheet. Server-side generation with a headless browser toolchain such as Puppeteer or `wkhtmltopdf` was rejected because it would have meant hosting yet another service purely to produce a document the

device can already render, and because on-device generation keeps complaint data on the user's device.

Device and media integrations. Evidence capture uses `expo-image-picker` to take a photograph or select one from the gallery, returning base64 data that is uploaded to Supabase Storage; `expo-file-system` manages generated report files; `expo-image` provides cached, progressive image rendering; `expo-secure-store` holds sensitive session material on the device; and `@react-native-async-storage/async-storage` persists the Supabase session and cached notification state so the user is not signed out between launches. Interface behaviour relies on **React Native Reanimated 4** with the worklets runtime for animations that run on the UI thread, **React Native Gesture Handler** for native-quality gestures, **React Native Safe Area Context** and **Screens** for correct native layout, and **React Native Keyboard Controller** to keep long complaint and registration forms usable while the keyboard is open. The analytics charts are composed from React Native primitives and the project's own theme tokens rather than from a heavyweight charting library, which kept the visualisations consistent with the rest of the interface and avoided a dependency that renders differently on web and native.

State management. Application state is managed with React Context providers and custom hooks (for authentication, complaints, notifications and analytics) rather than an external state library such as Redux or Zustand. Because Supabase is the single source of truth and each screen queries the data it needs, a global normalised client store would have added indirection without solving a problem the project actually has.

5.6 Development Tooling

Development uses the **Expo CLI** with the **Metro** bundler for the development server and bundling, **ESLint** with `eslint-config-expo` for static analysis, **TypeScript** path aliases (`@/*` → `src/*`) to keep imports stable as the file tree grows, and **Git** for version control. Database changes are tracked as versioned SQL migration files inside the repository, and the current schema and Row Level Security policies are maintained as living documentation alongside them, so the database can be rebuilt from scratch in a new Supabase project.

5.7 Consolidated Justification

Table 3 summarises, for each major requirement of the system, the technology adopted, the alternatives that were considered, and the reason for the final decision.

Table 3: Technology decisions and their justification

Requirement	Adopted	Alternatives	Reason for Choice
Cross-platform mobile client	Expo (React Native)	Native Kotlin/Swift; Flutter; bare React Native	One TypeScript codebase for Android, iOS and web; managed native modules remove manual linking for twenty-plus native dependencies; team already fluent in TypeScript
Role-separated navigation	Expo Router with route groups	Manually configured React Navigation	File-based routes mirror the four user modules; role guards live in per-group layouts; typed routes catch invalid navigation at compile time
Relational data with audit history	Supabase PostgreSQL	Firebase Firestore; self-hosted Express and PostgreSQL	Foreign keys, ENUM domains and transactional triggers express the ER diagram directly; the generated client SDK removes the need to host a separate API
Evidence image hosting	Supabase Storage	Cloudinary; Amazon S3	Same project, credentials and access-policy model as the database; public URLs are referenced straight from complaint rows
Event-driven notifications	PL/pgSQL triggers	Client-side writes; serverless cloud functions	Notifications are written in the same transaction as the state change, so they can never drift from the data, whichever client caused the change
Time-based notifications	pg_cron	External cron host; scheduled worker service	Runs next to the data it inspects, with no extra service to deploy or monitor
Complaint categorisation	Google Gemini Flash (multi-modal)	Fine-tuned BERT; TF-IDF classifier; other commercial LLMs	No labelled Bangladeshi complaint corpus exists; photographs and text are classified in a single request; free tier and low latency fit the submission flow

Requirement	Adopted	Alternatives	Reason for Choice
Duplicate detection	Gemini with an SQL candidate prefilter	Keyword or TF-IDF similarity; <code>pgvector</code> embeddings	Recognises semantically identical reports written in different words and languages, weighs location and recency, and returns a human-readable reason for administrator review
Out-of-band user notices	BulkSMSBD API	Twilio; transactional email; push notifications	Local eleven-digit number support and operator routing at local cost; reaches users who cannot yet sign in to receive an in-app notification
Report export	<code>expo-print</code> with <code>expo-sharing</code>	Server-side Puppeteer or <code>wkhtmltopdf</code>	Renders PDFs on the device, so no additional service is hosted and complaint data stays local
Client state	React Context with custom hooks	Redux Toolkit; Zustand	Supabase is the single source of truth; per-screen queries make a global normalised store unnecessary